

The Kaiju Lab, compressed

Shared participant instructions - every scenario, same steps

Contents

- Overview and rules
- Before the session
- Section 1: Discovery
- Section 2: Security and Responsible AI
- Section 3: Research, Plan, Implement
- Show-and-tell and close

Derived from The Kaiju Lab v1.0 (Sumer Verma, 7 October 2026). Capgemini internal. The sample application is MIT licensed; the scenario is fiction and contains no client material.

Kaiju Lab, compressed: shared instructions

Same steps for every scenario. Your scenario pack has the kaiju facts, the role cards and the slug. Where this document says <slug>, use the one from your pack.

One team of 2 to 4. Three sections, about 25, 20 and 25 minutes. Everyone touches discovery, security, Responsible AI and the research-plan-implement loop. Nobody finishes any of them. That is the point.

Each scenario has its own repo. It commits `.copilot-tracking/` on purpose so the team shares artifacts through pull requests. A real project keeps it ignored.

Rules

- Kaiju facts are fixed. Page 1 of your pack has everything you need; the coach holds the rest. Never invent them.
- Human facts are yours. Invent them in role, and label them.
- "I do not know" and "assume and label it" are good answers.
- Approve every permission prompt by hand. Never auto-approve.
- Nothing real goes in the workspace. Push only to your scenario repo.
- Never touch `main`. Your cohort works on `team/<cohort>`; every pull request targets that branch.

Before the session

Do these the week before. Blocked on any of them, tell the facilitator before the day.

- 1 Claude Code installed with the hve-core plugin. `/plugin` lists it; typing `/hve-core:` shows commands.
- 2 Node 22. In the repo: `npm install`, then `npm run dev`. Four kaiju visible on the map; the roster shows six.
- 3 `git push` works from your terminal to your scenario repo. Prove it with a scratch branch.
- 4 Read your card. One card per person. Read page 1 of the pack. Nothing else.
- 5 Pick a reasoning model by hand. Automatic selection gives shallow coaching.

On the day, before typing:

- Open Claude Code **in the repo folder**, not a parent. Run `pwd` and check. An agent writes wherever it is.
- Start from your cohort's branch, never `main`: `git fetch`, then `git checkout team/<cohort>`. The facilitator tells you the cohort name.
- `/clear` at the start of every section. A prompt typed into the wrong agent produces confident nonsense.
- Always type the agent command, never the prompt command. `/hve-core:dt-coach`, `/hve-core:security-planner`, `/hve-core:rai-planner-agent`, `/hve-core:rpi-agent`. The prompt commands such as `security-plan-from-prd`, `rai-capture` and `rpi` fail with "needs the role loaded first". Known defect in the Claude Code conversion.
- Open `.copilot-tracking/` in your file explorer. Watch files appear. That folder is what gets reviewed, not your chat.

Section 1: Discovery

Goal. One role-play interview per person against the fixed outcome on page 1 of your pack, merged into one folder. Then one synthesis that produces a PRD, a technology-neutral diagram and an Azure overlay.

Everyone. `/clear`, then `/hve-core:dt-coach`, then paste the prompt on your card. The coach reads the shared scope and asks five questions. Answer from inside your card's "can see" window and "your scope". Outside them, say "I do not know" or "that is not my call, ask X". For fictional detail, "assume and label it". When the coach writes your interview file, read it once.

Then: `git checkout -b <cohort>/<your-role>`, `commit`, `push`, open a pull request with base `team/<cohort>`. Not main.

Lead. Merge all pull requests into `team/<cohort>`. Fewer than four people: copy each unassigned card's folder from your pack's fallback/ into `.copilot-tracking/dt/` first. `git pull`. Then `/clear`, `/hve-core:dt-coach`, and:

```
Project <slug>. Read the shared scope under .copilot-tracking/dt/<slug>/ and
every interview under .copilot-tracking/dt/<slug>*/. Restate the outcome in
light of the interviews, keeping the original beside it. Summarise the problem as
if hearing it for the first time. List what you still need before a PRD. Then
draft the PRD with non-functional requirements, close gaps with labelled
assumptions, and state what it rests on. Then a technology-neutral diagram as
Mermaid. Then an opinionated Azure overlay with the minimum services, rejected
alternatives recorded, and a latency budget. Write everything under
.copilot-tracking/dt/<slug>/. Run without stopping. Label all interviews
role-play.
```

Everyone else, while it runs. Read the pre-seeded interviews. Find the one gap they all describe from different sides.

Done when. PRD, diagram and overlay exist. Each person can name one coach question they did not expect.

If. Coach restarts scope: "scope is done, read coaching-state.md". Coach declares no software needed: "a software tool is expected, continue". Coach offers the canonical deck: "no, do not ask again".

Section 2: Security and Responsible AI

Goal. A severity-ranked threat model against the team's PRD, and a Responsible AI assessment of the app's Last Stand decision. Two terminals, two drivers, in parallel.

Everyone, first. Write three security findings you predict the planner will raise. Paper. Keep it.

Security driver. /clear, then /hve-core:security-planner, then:

Use the security planner to plan the production security for this solution. PRD and architecture are under .copilot-tracking/dt/<slug>/. Scoping answers are in security-scoping-answers.md in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Commit only, never push.

Approve prompts as they come. Allowed: it answers its own scoping. Not allowed: "assume I confirm everything". When done, ask for the plan summary by severity, counts per level, Critical items one line each, and its recommended next planner. Compare the Critical list with the paper predictions.

RAI driver. /clear, then /hve-core:rai-planner-agent, then:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name single-save-decision. Imagine the scoping answers, label them, run to completion unless blocking.

Read the two opening observations before replying. Do not argue the risk down; "it is only a game" is cheating. When done, read the go/no-go and the minimum conditions.

Done when. Both summaries exist. Team can name one Critical threat nobody predicted and one RAI minimum condition an engineer would have skipped.

Section 3: Research, Plan, Implement

Goal. Take the Last Stand coin flip to a governed feature. Same feature for every scenario. Research and plan are pre-seeded from a facilitator run under `.copilot-tracking/research/`, `plans/` and `reviews/plans/`. The team starts at the plan gate, the human decision in this loop.

Reviewer, read aloud. Open

`.copilot-tracking/plans/2026-10-07/single-save-decision-governance-plan.md`. Read the executive summary and the three agent proposals PD1 to PD3 under "Planning Decisions and Feedback".

Team, decide. Each proposal: accept, reject, or accept with a condition. The usual argument is PD2, whether a commander must give a reason when overriding the recommendation. Decide it.

Lead, amend the file. Under each PD row, change the status and add one line of rationale. If a decision adds behaviour, add a functional requirement and a test case. Save. Correct the artifact, not the chat.

Lead, implement. `/clear`, `/hve-core:rpi-agent`, then:

```
Resume task single-save-decision-governance at the plan gate. The plan file has
been amended by the team; re-read it, then implement Story 1. Validation is the
existing test, lint and build scripts. No backend, no tracker, no cloud tenant:
mock or defer and log the gap. Keep tracking paths out of code and commits.
```

Approve edits as they come. It implements before you approved? Cancel, restart from the plan file.

Lead, review. When implement stops: `/hve-core:rpi-review`. Read the verdict. Findings route to the phase that owns them, not to a patch.

If time runs out. Implement and review are homework. Bring the verdict to the next session.

Done when. Plan file carries the team's decisions. Implement has started or finished. Review verdict read, or scheduled.

Show-and-tell and close

Three minutes per team. Four items, one sentence each.

- 1 One coach question nobody expected.
- 2 One Critical threat nobody predicted.
- 3 One plan amendment and why.
- 4 The review verdict, or where implement stopped.

Room names gaps, not compliments.

Closing questions.

- Which finding would your team have missed without the planner?
- The review found zero defects and still refused acceptance. Would you have shipped?
- What broke today, and would it break on a client machine?

Afterwards. Commit your branch. Code and tracking artifacts both, in your scenario repo only. The agent writes the commit message. Never push anywhere else.